Document number: P0356R4
Date: 2018-10-04
Audience: Library Working Group
Reply-to: *Tomasz Kamiński <tomaszkam at gmail dot com>*

Simplified partial function application

1. Introduction

This document proposes and introduction of the new library functions for performing partial function application and act as replacement for existing `std::bind`.

This paper addresses LEWG Bug 40: variadic bind.

This paper was positively reviewed by LEWG and forwarded to LWG for inclusion into IS during Albuquerque meeting.

2. Revision history

2.1. Revision 0

This proposal is successor of the N4171: Parameter group placeholders for bind, that was proposing an extension of the existing `std::bind` to introduce new class of placeholders that would presents group of call arguments instead of one.

In this paper `bind_front` is proposed as alternative, that allow user to provide values that will be passed as first arguments to stored callable. The author believes that this solution is in-line with LEWG recommendation for the original paper, that suggested to introduce only `_all` placeholder.

2.2. Revision 1

- Removed `bind_back` function from the scope of the proposal per LEWG guidance. This decision was motivated by lack of compelling use cases for this function.
- Described problems caused by silent dropping of output parameters in 4.1. No arbitrary argument rearrangements section.
- Included proposed wording and feature testing macro.
- Corrected uses of `std::result_of` in implementation.

2.3. Revision 2

- Updated wording to N4687 (C++ Working Draft, 2017-07-30).
- Call wrappers now require propagation of the `noexcept`/`constexpr` specification.
- Replaced uses of deprecated `result_of` with `invoke_result`.
- Reworked wording to reffer to "initialization" instead of "construction" of objects.
- Replaced reference to "objects" to "entities" to cover binding references.

2.4. Revision 3

- Paper now targets LWG after positive LEWG review.
- Corrected uses of shall in [func.require] paragraph.
- Clarified that call wrappers produced by same specialization are same (addressing LWG issue 3023).
- Reverted deprecation wording for `bind` per LEWG guidance.

2.5. Revision 4

- Updated wording to N4762 (C++ Working Draft, 2018-07-07).
- Updated reference to P0318: `unwrap_ref_decay` and `unwrap_reference`.
- Applied wording fixes from Walter E. Brown.
- Removed "shall be a callable object" requirement.
- Removed definition of *expression-equivalent* (accepted as part of P0898R3: Standard Library Concepts).
- Split "Requires" elements into "Mandates" and "Expects", per P0788R3: Standard Library Specification in a Concepts and Contracts World.
- Changed links to use https://wg21.link.

3. Motivation and Scope

This paper proposes `bind_front` function for partial function application for first arguments of the function. In other worlds `bind_front(f, bound_args...)(call_args...)` is equivalent to `std::invoke(f, bound_args..., call_args...)`.

It is worth to notice that proposed function provides both superset and subset of existing `std::bind` functionality: their support passing variable number of arguments, but does not allow arbitrary reordering or removal of the arguments. However author believes that proposed simplified functionality covers most of use cases for original `std::bind`.

3.1. Passing arguments

Let us consider an example task of writing the functor that will invoke `process` method on copy of `strategy` object:

```
struct Strategy { double process(std:string, std::string, double, double); };
std::unique_ptr<Strategy> createStrategy();
```

Firstly, such functor should not cause any additional overhead caused by passing the argument values from the call side to the stored callable. To achieve desired effect in case of lambda based solution, we can use forwarding reference in combination with variadic number of arguments:

```
[s = createStrategy()] (auto&&... args) { return s->process(std::forward<decltype(args)>(args)...); }
```

In case of the functors produced by `std::bind`, perfect forwarding is used by default for all the call arguments that are passed in place of placeholders, so same effect may be achieved using:

```
std::bind(&Strategy::process, createStrategy(), _1, _2, _3, _4)
```

However use of named placeholder requires user to decide on specific number of arguments passed to the function, so it does not support variadic functors and require wrapper code to be adjusted each time when the number of arguments accepted by target callable is changed. In addition to that it leads to more subtle and hard to spot problems presented in 4.1. No arbitrary argument rearrangements section of this paper.

In contrast in case of proposed `bind_front` function, all arguments provided on the call side are forwarded to the callable. As consequence the user is not required to manually write boilerplate code for perfect forwarding nor are exposed to potential errors caused by use of placeholders:

```
bind_front(&Strategy::process, createStrategy())
```

3.2. Propagating mutability

In our previous example the strategy object was stored in the callable indirectly by the use of the smart pointer, so it mutability was not affected by the functor. However in case of storing object by value we would like to propagate constness from the functor. That means for each of the following declarations:

```
auto f = [s = Strategy{}] (auto&&... args) { return s.process(std::forward<decltype(args)>(args)...); }; // 1
auto f = std::bind(&Strategy::process, Strategy{}, _1, _2, _3, _4); // 2
auto f = bind_front(&Strategy::process, Strategy{}); // 3
```

Invocation on mutable version of the functor (`f`) shall invoke `process` method on mutable object (call well-formed), however in case of const qualified one (`std::as_const(f)`) `process` method shall be invoked on const object (call ill-formed). This functionality is supported both by existing `std::bind` (2) and proposed `bind_front` (3), however it is not in case of the lambda (1). This is caused by the fact that closure created by the lambda has only one overload of the call operator that is const qualified by default.

As consequence, in case of use of lambda based solution, user must decide if he want to pass each object as const and allow only calls on const object, by use of:

```
[s = Strategy{}] (auto&&... args) { return s.process(std::forward<decltype(args)>(args)...); };
```

Or allow modification of stored objects, but limits calls to non-const functors only:

```
[s = Strategy{}] (auto&&... args) mutable { return s.process(std::forward<decltype(args)>(args)...); };
```

Same problems may occurs in situation when stored function supports both const and mutable calls via appropriate overloads of `operator()`. For example in case of following class:

```
struct Mapper
{
  auto operator()(int i, int j) -> std::string& { return _mapping[{i, j}]; }
  auto operator()(int i, int j) const -> std::string const& { return _mapping[{i, j}]; }

private:
  std::map<std::pair<int, int>, std::string> _mapping;
};
```

Functors produced by `std::bind(Mapper{}, 10, _1)` and `bind_front(Mapper{}, 10)` will call both const and non-const overloads, depending on their qualification. While in case of lambda, user will need to decide to support only one of them, by using one of:

```
[m = Mapper{}](int i) -> std::string const& { return m(10, i); }
[m = Mapper{}](int i) mutable -> std::string& { return m(10, i); }
```

3.3. Preserving return type

The reader may notice that lambda functions used in previous section, are explicitly specifying their return type. This is caused by the fact that lambda is using the `auto` deduction for the return type as default. As consequence the following slightly changed declaration would return `std::string` object by value:

```
auto fc = [m = Mapper{}](int i) { return m(10, i); };
auto fm = [m = Mapper{}](int i) mutable { return m(10, i); };
```

Such slight change of code may lead to various changes in the behaviour of the program. Firstly additional copy construction will be invoked, if the object returned by the lambda is captured by reference:

```
auto const& s1 = fc(2);
auto const& s2 = fm(2);
```

Secondly, the lifetime of object returned from the functor will not longer be tied to the lifetime of the `Mapper` object, which may lead to creation of dangling references:

```
auto f = [m = Mapper{}](int i) { return m(i, 10); };
std::string* ps = nullptr;
{
  auto const& s = f(2);
  ps = &s;
}
// *ps is dangling
```

Lastly in case of the mutable version of functor, changing the result of the invocation modifies temporary, not the mapped value:

```
fm(2) = "something";
```

To avoid such problems we may use `decltype`-based return type deduction, as it is done in case of `std::bind` and proposed `bind_front`:

```
[m = Mapper{}](int i) -> decltype(auto) { return m(i, 10); }
```

3.4. Preserving value category

If we consider following example implementation of the functor that performs memoization of the expensive to compute function `func`:

```
struct CachedFunc
{
  std::string const& operator()(int i, int j) &
  {
     key_type key(i, j);

     auto it = _cache.find(key);
     if (it == _cache.end())
       it = _cache.emplace(std::move(key), func(i, j)).first;

     return it->second;
  }

private:
  using key_type = std::pair<int, int>;
  std::map<key_type, std::string> _cache;
};
```

As we can see `CachedFunc::operator()` is using reference qualification to limit valid calls only to lvalues. Use of this qualification allows us to avoid dangling reference problems, in situation when reference returned by temporary `CachedFunc` object would be used after its destruction. In addition it signals that use of `CachedFunc` makes sense only in situation when it is invoked multiple times and for one-shot invocation invoking `func` directly is more optimal solution.

As in case of the `const` propagation, we would like to preserve/propagate value category from the functor to stored callable. That means that for the following declarations:

```
auto f = [cache = CachedFunc{}] (int j) mutable -> std::string& { return cache(10, j); }; // 1
auto f = std::bind(CachedFunc{}, 10, _1); // 2
auto f = bind_front(CachedFunc{}, 10); // 3
```

Invocation on the lvalue (`f(1)`) shall perform call on the lvalue of `CachedFunc` and be well-formed, while invocation on the rvalue (`std::move(f)(1)`) shall lead to call on the rvalue and be ill-formed.

Out of discussed options, only proposed `bind_front` (3) functions are preserving value category. In case of existing `std::bind` and lambda solutions, the call is always performed on lvalue regardless of the category of function object, and essentially bypass reference qualification.

Same problems also occurs in case of the bound arguments, even if the callable does not differentiate between calls on lvalues and rvalues. For example if we consider following function declarations

```
void foo(std::string&);
auto make_bind(std::string s)       { return std::bind(&foo, s); }
auto make_lambda(std::string s)     { return [s] { return foo(s); }; }
auto make_bind_front(std::string s) { return bind_front(&foo, s); }
```

Invocations in the form `make_bind("a")()` and `make_lambda("a")()` are well-formed and are invoking function `foo` with lvalue reference to de-facto temporary string (member of temporary functor). In case of proposed functions, value category of the functor also affects stored arguments and corresponding call `make_bind_front("a")()` is ill-formed.

3.5. Supporting one-shot invocation

Lack of propagation of the value category in existing partial function application solutions, prevents them from supporting functors that allows one-shot invocation via rvalue qualified call operator. As consequence for the following declarations:

```
struct CallableOnce
{
  void operator()(int) &&;
};

auto make_bind(int i)       { return std::bind(CallableOnce{}, i); }
auto make_lambda(int i)     { return [f = CallableOnce{}, i] { return f(i); }; }
auto make_bind_front(int i) { return bind_front(CallableOnce{}, i); }
```

Only the invocation `make_bind_front(1)()` is well formed, as the other two (`make_bind(1)()` and `make_lambda(1)()`) leads to unsupported call on the lvalue of `CallableOnce`.

Using a lambda expression it would be possible to work around the problem by explicit use of the `std::move`:

```
[f = CallableOnce{}, i] { return std::move(f)(i); }
```

However above code is forcing calls on rvalue of `CallableOnce`, even if lvalue functor is invoked. As consequence multiple calls may be performed on single instance of `CallableOnce` class.

It is also worth to notice, that one-shot callable functors may also be produced as a result on binding an move-only type. For example in situation when we want to bind arguments to a function `consume` that accepts `std::unique_ptr<Obj>` by value:

```
struct ConsumeBinder
{
  ConsumeBinder(std::unique_ptr<Obj> p)
    : ptr(std::move(p))
  {}

  void operator()() &&
  { return consume(std::move(ptr)); }

private:
  std::unique_ptr<Obj> ptr;
};
```

In addition support for one-shot invocation is leading to improved performance. For example let's consider situation, when we want to bind a vector `v` as the first argument to the following function:

```
void bar(std::vector<int>, int)
```

Depending on the scenario, at the point of the call of the bind-wrapper (`bw`) that we will create, we may want to:

- move stored vector to `bar` function, if `bw` will be called only once (one-shot)
- copy stored vector to `bar` function, if `bw` will be called multiple times

Proposed `bind_front` function support both scenarios, via rvalue and lvalue overloads of call operator. Consequently if `bw` is created using `bind_front(&bar, v)`:

- `std::move(bw)(10)` will move stored vector (pass as rvalue reference)
- `bw(10)` will copy stored vector (pass as lvalue reference)

3.6. Preserving exception specification

In contrast to existing callable wrappers, proposed `bind_front` function is required to preserve exception specification of the underlining call operator. This follows recent additions to the language, that make the exception specification part of type system and `std::invoke` conditionally `noexcept` - combination of this changes allowed `noexcept` specification to be preserved for invocation of function pointers.

Finally, this guarantee is extended also for `not_fn` function via alternative wording included in the paper.

3.7. No compile time evalutation support

Wording included in the paper, requires that the invocation of the functor created via `bind_front` and `not_fn` will support compile time evaluation, when underlining expression is compile time constant. However this requirement is latent, as the underlining `std::invoke` function is not `constexpr` qualified.

Secondly, the author believes that introduction of the `constexpr` support for the `std::invoke` is outside of the scope of this paper, as under current direction of CWG issue 1581 such change would be breaking (example clang bug report).

4. Design Decisions

The section provides rationale for avoiding usage existing `std::bind` even in the situation when proposed new functions does not strictly supersede its functionality.

4.1. No arbitrary argument rearrangements

In contrast to the `std::bind` proposed `bind_front` does not support rearrangements or dropping of the call arguments, that was supported by `std::bind`.

Firstly, handling of the placeholder was requiring a large amount of the meta-programming, to only determine types and values of the argument that will be actually passed to stored callable. However in this case required complexity of implementation is not only affecting the vendors, but also leads to unreadable error message produced to the user.

Secondly, it allow wrapper created from `std::bind` to silently drop arguments that are not referenced by the placeholders. This functionality may seem to be unharmful, but in case of output parameters it prevents certain range of bugs from being detected as compile time. For example following code will silently ignore potential error passed to the `callback` and will cause program to loop infinitely if such error is reported:

```
struct InputStream
{
  void read_async(std::size_t count, std::vector<char>& out,
                  std::function<void(std::error_code)> callback);
};
```

```
class DataReader
{
   /* rest of interface */

   void read_remaining()
   {
      stream.read_async(expected - content.size(), content,
                        std::bind(&DataReader::part_done, this));
   };

   void part_done()
   {
     if (content.size() < expected)
       read_remaining();
   }

   InputStream stream;
   std::size_t const expected;
   std::vector<char> content;
};
```

Finally, it allows user to write a code that will pass same value to the function multiple times, by repeated use of single placeholder, which in case of use of move semantics may lead to passing of unspecified values as arguments. For example values of first and second argument are unspecified in case of following invocation:

```
auto f = std::bind(&Strategy::process, createStrategy(), _1, _1, _2, _2);
f(std::string("some_string"), 1);
```

Occurrence of such kind of problems depends both of bound arguments passed to the bind and once that are provided on the call side. As consequence addressing them, would require introduction of another type of placeholder that would pass rvalues as const rvalue reference or additional compile-time logic that will detect duplicated arguments and modify their value category. However both solutions would only increase, already large, complexity of implementation and use.

4.2. No nested *bind expressions*

The proposed function do not give any special meaning to the nested *bind expressions* (functors produced by `std::bind`) and they are passed directly to the stored callable in case of the invocation.

Firstly, in the author's opinion, use of nested bind leads to unreadable code that are clearly improved by being replaced with custom functor, especially in situation when such functor can be created in place using lambda expression.

Secondly, special treatment of nested *bind expressions* and placeholders hardens the reasoning about behaviour of bind expression, by leading to the situations when `std::bind(f, a, b, c)()` is not invoking `f(a, b, c)`, despite the user intent. This may occur in situation when type of values passed to `std::bind` are not known by the programmer at point of binding:

```
struct apply_twice
{
  template<typename F, typename V>
  auto operator()(F const& f, V const& v) const
    -> decltype(f(f(v)))
  { return f(f(v)); }
};

template<typename F>
auto twicer(F&& f)
{ return std::bind(apply_twice{}, std::forward<F>(f), _1); }

double cust_sqrt(double x) { return std::sqrt(x); }
double cust_pow(double x, double n) { return std::pow(x, n); }
```

Invocation of `twicer(&cust_sqrt)(16)` is valid and return 2, while `twicer(std::bind(&cust_pow, _1, 2))(2))` is invalid.

4.3. Fixing `std::bind`

Additional motivation for introduction of then new function, is that fixing the problems mentioned above in `std::bind` would require introduction of breaking changes to the existing codebase. Furthermore such changes would not only take verbose form, when previously valid code will no longer compile, but may also silently change its meaning, by selecting different overload of underlining functor. The author believes that in such case introduction of new functions would be required anyway.

5. Impact On The Standard

This proposal has no dependencies beyond a C++14 compiler and Standard Library implementation.

Nothing depends on this proposal.

6. Proposed Wording

The proposed wording changes refer to N4762 (C++ Working Draft, 2018-07-07).

6.1. Definition of *perfect forwarding call wrapper*

This section introduces a *perfect forwarding call wrapper* concept, that groups a set of common requirements for the call wrapper for a stateful callable(s) and collection of bound arguments. This concept is later used to simplify the specification of the proposed `bind_front` and existing `not_fn` function. In addition it can be extended to cover `overload` function (proposed in P0051: C++ generic `overload` function), by allowing call wrapper to hold multiple target objects.

The definition of the *perfect forwarding call wrapper* requires the implementation to guarantee that the invocation of the wrapper object will be core constant expression and/or `noexcept` depending on the invoked call expression. However, due the fact that both proposed functors are defined in terms of the `std::invoke` function, this only enforce implementation to provide conditional `noexcept` specification for `operator()` of the wrapper.

In addition to avoid potential confusion, existing *forwarding call wrapper* is renamed to *argument forwarding call wrapper*, to better reflect its functionality.

Apply following changes to paragraph [func.def] Definitions:

> A *call wrapper type* is a type that holds a callable object and supports a call operation that forwards to that object.
>
> A *call wrapper* is an object of a call wrapper type.
>
> A *target object* is the callable object held by a call wrapper.
>
> A call wrapper type may additionally hold a sequence of objects, references to functions and references to objects, that may be passed as arguments to the target object. These entities are collectively referred to as *bound argument entities*.
>
> The held target object and bound argument entities of the call wrapper are collectively referred to as *state entities*.

Apply following changes to paragraph [func.require] Requirement:

> Every call wrapper ([func.def]) ~~shall be~~is *Cpp17MoveConstructible*. An *argument forwarding call wrapper* is a call wrapper that can be called with an arbitrary argument list and delivers the arguments to the wrapped callable object as references. This forwarding step ~~shall~~ ensures that rvalue arguments are delivered as rvalue references and lvalue arguments are delivered as lvalue references. A *simple call wrapper* is an argument forwarding call wrapper that is *Cpp17CopyConstructible* and *Cpp17CopyAssignable* and whose copy constructor, move constructor, and assignment operator do not throw exceptions. [Note: In a typical implementation argument forwarding call wrappers have an overloaded function call operator of the form
>
> ```
> template<class... UnBoundArgs>
> R operator()(UnBoundArgs&&... unbound_args) cv-qual;
> ```
>
> — end note].
>
> A *perfect forwarding call wrapper* is an argument forwarding call wrapper that propagates its state entities to the underlying call expression. This propagation step ensures that the state entity of type `T` is delivered as `T cv&` when the call is performed on an lvalue of the call wrapper type and as `T cv&&` otherwise, where *cv* represents *cv*-qualifiers of the call wrapper and where *cv* shall be neither `volatile` nor `const volatile`.
>
> A perfect forwarding call wrapper of type `G` with a *call pattern* `cp` ensures that a postfix call expression performed on an expression of type of potentially const qualified reference to `G` is expression-equivalent ([defns.expression-equivalent]) to an expression `e` determined as follows: every occurrence of the name of the argument of the call wrapper or one of its state entities in `cp` is replaced with an expression of reference type determined by the corresponding propagation rules.
>
> The copy and move constructors of the perfect forwarding call wrapper have the same apparent semantics as the implicitly-defined memberwise operation performed for its state entities ([class.copy]) [Note: This implies that copy/move constructors have the same *exception-specification* as corresponding implicit definition and they are declared as `constexpr` if corresponding implicit definition would be considered to be `constexpr` - end note.]
>
> Perfect forwarding call wrappers have the same type, if their call patterns and types of their corresponding state entities are the same.

Replace all references to forwarding call wrapper with argument forwarding call wrapper in [func.bind.bind] Function template `bind`.

6.2. Wording for `bind_front`

After the declaration of `not_fn` in the section [functional.syn] (Header `<functional>` synopsis), add:

```
  // [func.bind_front], binders
  template <class F, class... Args> unspecified bind_front(F&&, Args&&...);
```

After section [func.not_fn] Function template `not_fn`, insert a new section.

Function template `bind_front` **[func.bind_front]**

```
  template <class F, class... Args>
    unspecified bind_front(F&& f, Args&&... args);
```

In the text that follows:

- `g` is a value of the result of a `bind_front` invocation,
- `FD` is the type `decay_t<F>`,
- `fd` is a target object of `g` ([func.def]) of type `FD` initialized with initializer `(std::forward<F>(f))`,
- `BoundArgs` is a pack of types equivalent to *`DECAY_UNWRAP`*`(Args)...`,
- `bound_args` is a pack of bound argument entities of `g` ([func.def]) of types `BoundArgs...` initialized with initializers `(std::forward<Args>(args))...` respectively,
- `call_args` is an argument pack used in a function call expression of `g`,

where *`DECAY_UNWRAP`*`(T)` is determined as follows: Let `U` be `decay_t<T>`. Then *`DECAY_UNWRAP`*`(T)` is `X&` if `U` equals `reference_wrapper<X>`, otherwise *`DECAY_UNWRAP`*`(T)` is `U`.

Mandates: `conjunction_v<is_constructible<FD, F>, is_move_constructible<FD>, is_constructible<BoundArgs, Args>..., is_move_constructible<BoundArgs>...>` shall be true.

Expects: `FD` shall satisfy the requirements of *Cpp17MoveConstructible*. For each `Ti` in `BoundArgs`, if `Ti` is an object type, `Ti` shall satisfy the requirements of *Cpp17MoveConstructible*.

Returns: A perfect forwarding call wrapper `g` with call pattern `invoke(fd, bound_args..., call_args...)`.

Throws: Any exception thrown by the initialization of the state entities of `g` ([func.def]).

Note: This wording can be further simplified by use of `unwrap_ref_decay_t` from P0318R1: `unwrap_ref_decay` and `unwrap_reference` paper.

6.3. Alternative wording for `not_fn`

This section presents an alternative wording for the `not_fn`, that aims to provide same guarantees as existing standard wording, without resorting to use of exposition only *`call_wrapper`* class. The major improvements over Library Fundamentals TS v2 wording, comes from using *perfect forwarding call wrapper* in definition of `not_fn(f)` return.

Change the section [func.not_fn] Function template `not_fn` to:

```
template <class F>
  unspecified not_fn(F&& f);
```

In the text that follows:

- `g` is a value of the result of a `not_fn` invocation,
- `FD` is the type `decay_t<F>`,
- `fd` is a target object of `g` ([func.def]) of type `FD` initialized with initializer `(std::forward<F>(f))`,
- `call_args` is an argument pack used in a function call expression of `g`.

Mandates: `is_constructible_v<FD, F> && is_move_constructible_v<FD>` shall be true.

Expects: `FD` shall satisfy the requirements of *Cpp17MoveConstructible*.

Returns: A perfect forwarding call wrapper `g` with call pattern `!invoke(fd, call_args...)`.

Throws: Any exception thrown by the initialization of `fd`.

7. Feature-testing recommendation

For the purposes of SG10, we recommend the macro name `__cpp_lib_bind_front` to be defined in the `<functional>` header.

Usage example:

```
struct Strategy { double process(std:string, std::string, double, double); };

auto bind_to_process(std::unique_ptr<Strategy> ptr)
{
#if __cpp_lib_bind_front
  return std::bind_front(&Strategy::process, std::move(ptr));
#else
  using namespace std::placeholders;
  return std::bind(&Strategy::process, std::move(ptr), _1, _2, _3, _4);
#endif
}
```

8. Implementability

Example implementation of proposed `bind_front`:

```
template<typename Func, typename BoundArgsTuple, typename... CallArgs>
decltype(auto) bind_front_caller(Func&& func, BoundArgsTuple&& boundArgsTuple, CallArgs&&... callArgs)
{
```

```
  return std::apply([&func, &callArgs...](auto&&... boundArgs) -> decltype(auto)
         {
           return std::invoke(std::forward<Func>(func), std::forward<decltype(boundArgs)>(boundArgs)..., std::forward<CallArgs>(callArgs)...);
         }, std::forward<BoundArgsTuple>(boundArgsTuple));
}

template<typename Func, typename... BoundArgs>
class bind_front_t
{
public:
  template<typename F, typename... BA,
           std::enable_if_t<!(sizeof...(BA) == 0 && std::is_base_of_v<bind_front_t, std::decay_t<F>>), bool> = true>
  explicit bind_front_t(F&& f, BA&&... ba)
    : func(std::forward<F>(f))
    , boundArgs(std::forward<BA>(ba)...)
  {}

  template<typename... CallArgs>
  auto operator()(CallArgs&&... callArgs) &
    noexcept(std::is_nothrow_invocable_v<Func&, BoundArgs&..., CallArgs...>)
    -> std::invoke_result_t<Func&, BoundArgs&..., CallArgs...>
  { return bind_front_caller(func, boundArgs, std::forward<CallArgs>(callArgs)...); }

  template<typename... CallArgs>
  auto operator()(CallArgs&&... callArgs) const &
    noexcept(std::is_nothrow_invocable_v<Func const&, BoundArgs const&..., CallArgs...>)
    -> std::invoke_result_t<Func const&, BoundArgs const&..., CallArgs...>
  { return bind_front_caller(func, boundArgs, std::forward<CallArgs>(callArgs)...); }

  template<typename... CallArgs>
  auto operator()(CallArgs&&... callArgs) &&
    noexcept(std::is_nothrow_invocable_v<Func, BoundArgs..., CallArgs...>)
    -> std::invoke_result_t<Func, BoundArgs..., CallArgs...>
  { return bind_front_caller(std::move(func), std::move(boundArgs), std::forward<CallArgs>(callArgs)...); }

  template<typename... CallArgs>
  auto operator()(CallArgs&&... callArgs) const &&
    noexcept(std::is_nothrow_invocable_v<Func const, BoundArgs const..., CallArgs...>)
    -> std::invoke_result_t<Func const, BoundArgs const..., CallArgs...>
  { return bind_front_caller(std::move(func), std::move(boundArgs), std::forward<CallArgs>(callArgs)...); }

private:
  Func func;
  std::tuple<BoundArgs...> boundArgs;
};

template<typename Func, typename... BoundArgs>
auto bind_front(Func&& func, BoundArgs&&... boundArgs)
{
  return bind_front_t<std::decay_t<Func>, unwrap_ref_decay_t<BoundArgs>...>{std::forward<Func>(func), std::forward<BoundArgs>(boundArgs)...};
}
```

To properly handle `std::reference_wrapper` in above code, we use `decay_unwrap` auxilary metafunction from [P0318R1: `unwrap_ref_decay` and `unwrap_reference`] paper.

```
template<typename T>
struct unwrap_ref_decay;

template<typename T>
struct unwrap_ref_decay<std::reference_wrapper<T>>
{
  using type = T&;
};

template<typename T>
struct unwrap_ref_decay
  : std::conditional_t<
      !std::is_same<std::decay_t<T>, T>::value,
      unwrap_ref_decay<std::decay_t<T>>,
      std::decay<T>
   >
{};

template<typename T>
using unwrap_ref_decay_t = typename decay_unwrap<T>::type;
```

9. Acknowledgements

Daniel Krügler offered tremendous amount of improvements for presented wording.

Marshall Clow for very helpful review of the paper wording.

Jonathan Wakely, Stephan T. Lavavej, Walter E. Brown, and Johel E. G. Peña offered many useful suggestions and corrections to the proposal.

Casey Carter has created and suggested use of expression-equivalent term in definition of *perfect forwarding call wrapper*.

Proposed runtime version of `bind_front` and `bind_back` are inspired by their compile time counterparts from Eric Niebler's Tiny Metaprogramming Library.

Special thanks and recognition goes to Sabre (http://www.sabre.com) for supporting the production of this proposal, and for sponsoring author's trip to the Oulu for WG21 meeting.

10. References

1. Chris Jefferson, Ville Voutilainen, "Bug 40 - variadic bind" (LEWG Bug 40, https://issues.isocpp.org/show_bug.cgi?id=40)
2. Mikhail Semenov, "Introducing an optional parameter for mem_fn, which allows to bind an object to its member function" (N3702, http://wg21.link/n3702)
3. Tomasz Kamiński, "Parameter group placeholders for bind" (N4171, http://wg21.link/n4171)
4. Jens Maurer, "P0012R1: Make exception specifications be part of the type system, version 5" (P0012R1, http://wg21.link/p0012r1)
5. Marshall Clow, "C++ Standard Library Issues Resolved Directly In Kona" (P0625R0, http://wg21.link/p0625r0)
6. William M. Miller, "C++ Standard Core Language Active Issues, Revision 97" (http://wg21.link/cwg1581)
7. Gonzalo BG, "[C++11/14] Body of constexpr function templates instantiated too eagerly in unevaluated operands" (Bug 23135, https://bugs.llvm.org/show_bug.cgi?id=23135)
8. Vicente J. Botet Escribá, "C++ generic `overload` function (Revision 1)" (P0051R1, http://wg21.link/p0051r1)
9. Vicente J. Botet Escribá, "`unwrap_ref_decay` and `unwrap_reference`" (P0318R1, http://wg21.link/p0318r1)
10. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4762, https://wg21.link/n4762)
11. Casey Carter, Eric Niebler, "Standard Library Concepts" (P0898R3, http://wg21.link/p0898r3)
12. Walter E. Brown, "Standard Library Specification in a Concepts and Contracts World" (P0788R3, http://wg21.link/p0788r3)